SafeRide School – Student Pickup & Drop-off Tracking

1. Prototyping

1.1 Product vision

SafeRide School is a mobile-first school transport safety platform that lets parents track the school bus in real time, receive instant updates when their child boards or alights, and contact the right people quickly during emergencies. It also helps drivers manage assigned routes and student attendance, while school administrators monitor operations, manage users, and respond to incidents.

1.2 Core user roles

Parents

- Track the bus in real time
- Receive pickup/drop-off notifications
- View student transport status
- Manage emergency contacts
- Receive SMS updates if the app is offline

Drivers

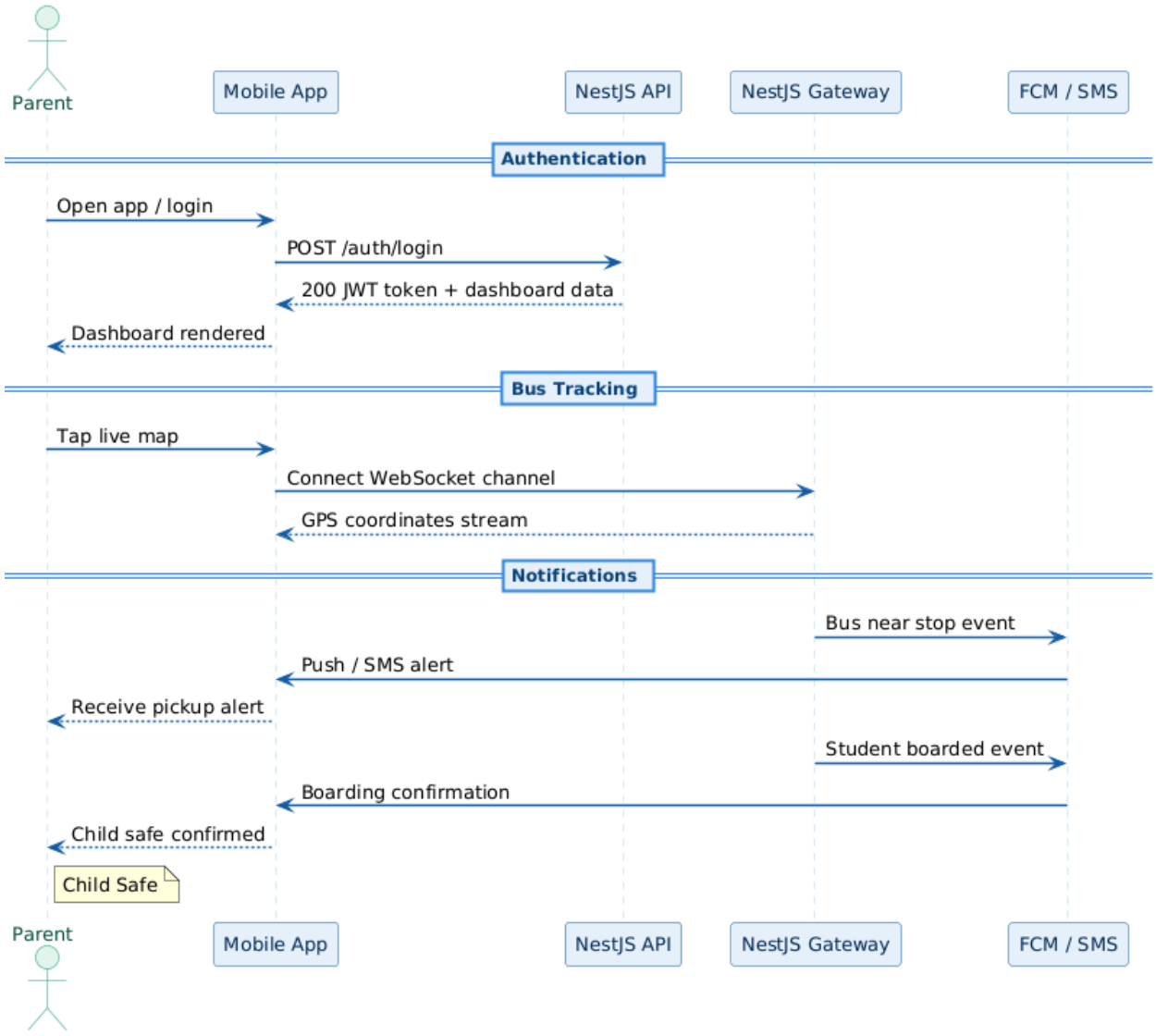
- Start and end trips
- Share live GPS location in background
- View student pickup list and route stops
- Mark student boarded/alighted
- Trigger emergency alert

School Admins

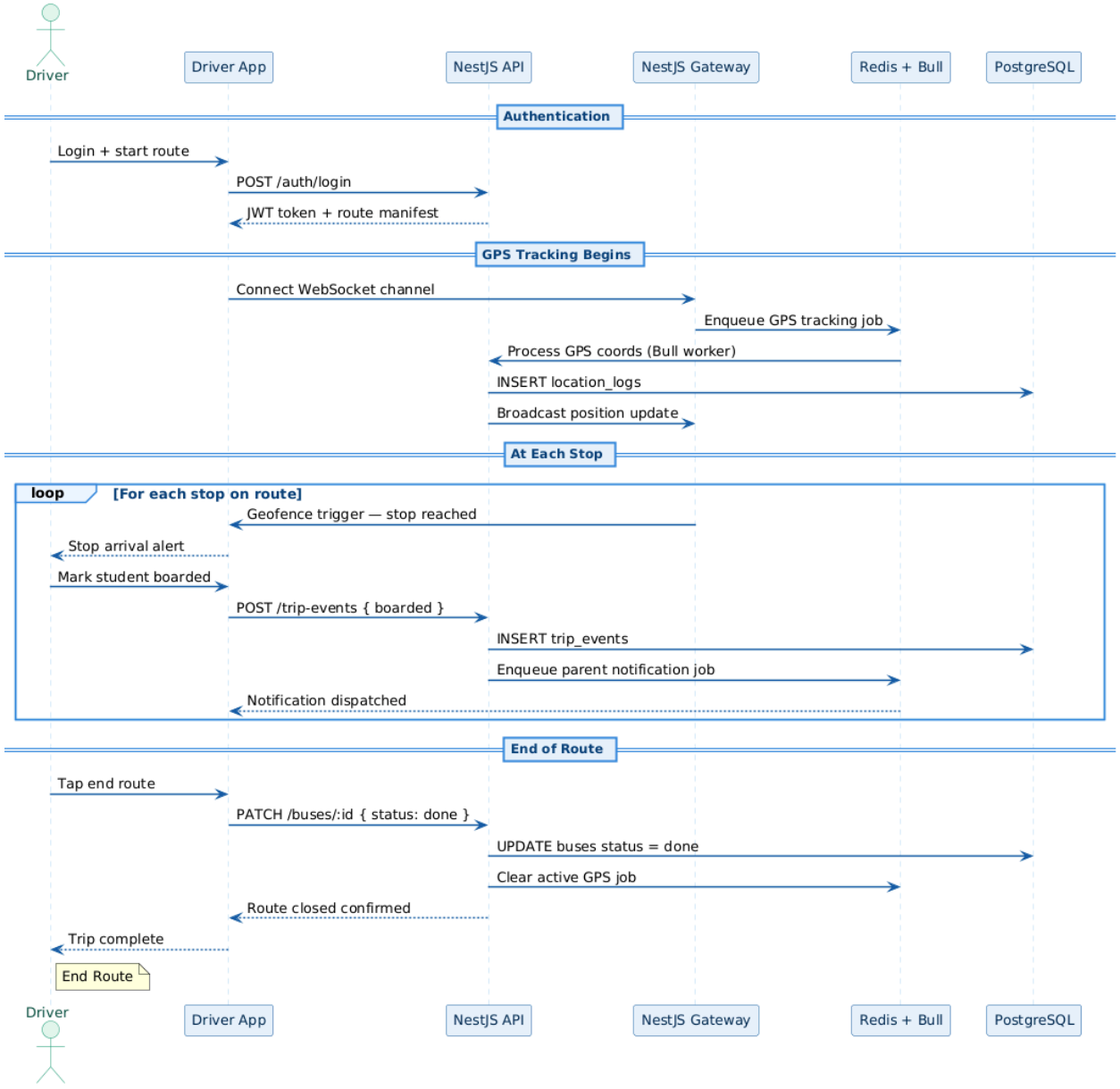
- Manage routes, buses, drivers, students, and guardians
- View live fleet status
- Monitor trip logs and incidents
- Configure notification rules
- Access reports and audit trails

1.3 Main user flows

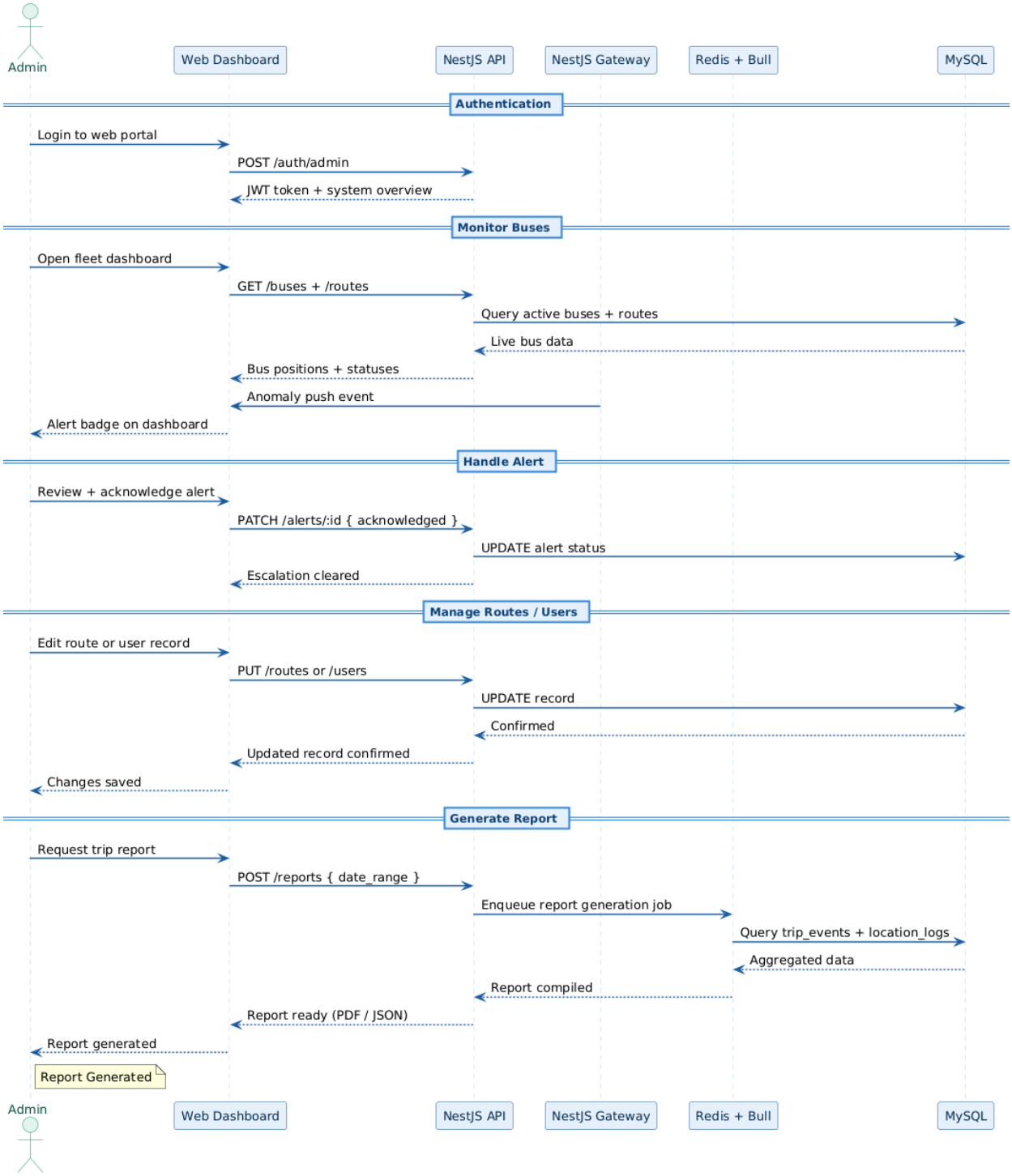
Parent flow



Driver flow


Driver
Driver App
NestJS API
NestJS Gateway
Redis + Bull
PostgreSQL
Authentication
Login + start route
POST /auth/login
JWT token + route manifest
GPS Tracking Begins
Connect WebSocket channel
Enqueue GPS tracking job
Process GPS coords (Bull worker)
INSERT location_logs
Broadcast position update
At Each Stop
loop
[For each stop on route]
Geofence trigger — stop reached
Stop arrival alert
Mark student boarded
POST /trip-events { boarded }
INSERT trip_events
Enqueue parent notification job
Notification dispatched
End of Route
Tap end route
PATCH /buses/:id { status: done }
UPDATE buses status = done
Clear active GPS job
Route closed confirmed
Trip complete
End Route
Driver
Driver App
NestJS API
NestJS Gateway
Redis + Bull
PostgreSQL

Admin flow


Admin
Web Dashboard
NestJS API
NestJS Gateway
Redis + Bull
MySQL
Authentication
Login to web portal
POST /auth/admin
JWT token + system overview
Monitor Buses
Open fleet dashboard
GET /buses + /routes
Query active buses + routes
Live bus data
Bus positions + statuses
Anomaly push event
Alert badge on dashboard
Handle Alert
Review + acknowledge alert
PATCH /alerts/:id { acknowledged }
UPDATE alert status
Escalation cleared
Manage Routes / Users
Edit route or user record
PUT /routes or /users
UPDATE record
Confirmed
Updated record confirmed
Changes saved
Generate Report
Request trip report
POST /reports { date_range }
Enqueue report generation job
Query trip_events + location_logs
Aggregated data
Report compiled
Report ready (PDF / JSON)
Report generated
Report Generated
Admin
Web Dashboard
NestJS API
NestJS Gateway
Redis + Bull
MySQL

1.4 Wireframe set

Screen 1: Splash / Onboarding

Purpose: Introduce app and guide users to role-based login.

Main elements:

- App logo and tagline: *Every journey, accounted for. Every child, safe.*
- Role selection shortcuts: Parent, Driver, Admin
- Login / Sign Up buttons
- Permissions prompt explanation for GPS, notifications, SMS, contacts

Screen 2: Login

Purpose: Secure entry for all user roles.

Main elements:

- Email / phone field
- Password / OTP option
- Role selector
- Forgot password link
- Login button

Screen 3: Parent Dashboard

Purpose: Show child transport status at a glance.

Main elements:

- Student card with profile photo and status badge
- Status examples: Waiting, Bus Approaching, Boarded, At School, Dropped Off
- Current trip progress bar
- Quick actions: Track Bus, Contact Driver, Emergency Contacts
- Latest notifications section

Screen 4: Live Bus Tracking Map

Purpose: Real-time tracking using GPS.

Main elements:

- Interactive map with bus marker
- Route polyline and stops
- Estimated arrival time
- Distance from current stop / home / school

- Refresh indicator and tracking status

Screen 5: Notifications Screen

Purpose: Display all transport alerts.

Main elements:

- Bus approaching alert
- Student boarded alert
- Student alighted alert
- Delay/route change alert
- SMS delivery status if push was unavailable

Screen 6: Driver Dashboard

Purpose: Help driver manage route and attendance.

Main elements:

- Assigned bus and route information
- Start Trip / End Trip button
- Student list by stop
- Mark Boarded / Mark Alighted buttons
- SOS / emergency button

Screen 7: Student Pickup Confirmation Screen

Purpose: Confirm student boarding or drop-off.

Main elements:

- Student name and photo
- Assigned stop
- Boarding status toggle
- Timestamp and GPS stamp
- Confirm action button

Screen 8: Admin Dashboard

Purpose: Monitor school transport operations.

Main elements:

- Overview cards: Active Trips, Delays, Students Onboard, Completed Trips
- Live map of buses
- Incident log

- Search and filter by route, driver, date

Link to wireframe:
https://www.figma.com/make/UoW8BylkOEfMMGfTeoUiiO/School-Bus-Tracking-App?t=BZHRkiJ4V7Lcn8cr-1

2. Specification

2.1 Functional Requirements

1. The system shall allow parents, drivers, and admins to log in securely.
2. The system shall support role-based access control.
3. The system shall show real-time bus GPS location to authorized parents and admins.
4. The driver app shall continuously send location updates in the background during active trips.
5. The system shall send push notifications when the bus is approaching a stop.
6. The system shall send push notifications when a student boards the bus.
7. The system shall send push notifications when a student alights from the bus.
8. The system shall send SMS fallback notifications when push delivery is unavailable or the parent device is offline.
9. The driver shall be able to mark students as boarded or alighted.
10. The system shall record timestamps for boarding and alighting events.
11. The system shall store trip history for each student and route.
12. Parents shall be able to manage emergency contacts from within the app.
13. Admins shall be able to create and manage routes, buses, stops, and users.
14. Admins shall be able to monitor live trip status and route progress.
15. The system shall generate incident and attendance logs.
16. The system shall support manual alerting by drivers in emergencies.

2.2 Non-functional requirements

Performance

- GPS updates should refresh every 5 to 10 seconds during active trips.
- Parent dashboard should load in under 3 seconds on standard mobile data.
- Notifications should be delivered with minimal delay.

Reliability

- GPS tracking should resume automatically after temporary connectivity loss.
- SMS fallback should handle notification failure scenarios.

Security

- All user data shall be encrypted in transit.

- Sensitive data such as guardian details and student records shall be protected.
- Role-based permissions shall restrict access to authorized information only.
- Authentication should support strong passwords and session management.

Usability

- The interface shall be simple enough for non-technical parents and drivers.
- High-priority alerts shall be visually prominent.
- Key actions shall be reachable within one or two taps.

Scalability

- The system should support multiple schools, routes, and buses.
- The backend should scale to handle concurrent location updates and notifications.

Maintainability

- Code should use modular architecture.
- Logging and error reporting should support debugging.

2.3 Use case table

Use Case	Actor	Goal	Preconditions	Main Success Scenario	Outcome
Log in	Parent / Driver / Admin	Access app securely	User account exists	User enters credentials and is authenticated	Role-based dashboard opens
Track bus	Parent	View bus location in real time	Student is assigned to an active route	Parent opens tracking map and sees live bus marker	Parent monitors ETA and movement
Start trip	Driver	Begin assigned route	Driver is authenticated and assigned a route	Driver taps Start Trip	Trip becomes active and GPS tracking starts
Mark student boarded	Driver	Confirm pickup	Trip is active and student is assigned to stop	Driver selects student and confirms boarding	Status updates and notification is sent
Mark student alighted	Driver	Confirm drop-off	Student is already boarded	Driver confirms drop-off	Status updates and notification is sent

Receive alert	Parent	Stay informed	Student event occurs	System sends push or SMS alert	Parent receives transport update
Manage emergency contacts	Parent	Keep contact list updated	Parent is logged in	Parent adds or imports emergency contacts	Contacts are saved for alerts/escalation
Monitor fleet	Admin	Supervise operations	Admin is authenticated	Admin opens live dashboard	Admin sees active routes, buses, and incidents
Manage routes	Admin	Configure transport operations	Admin is authenticated	Admin creates or edits routes/stops	Updated route assignments are saved
Trigger emergency alert	Driver	Report urgent issue	Active trip exists	Driver taps emergency alert	Admin and defined contacts are notified

3. Architecture

3.1 High-level system architecture

Mobile apps

- Parent mobile app
- Driver mobile app
- Admin mobile app or web dashboard

Backend services

- Authentication service
- User and role management service
- Trip management service
- GPS tracking service
- Notification service
- SMS gateway integration
- Emergency contact service
- Reporting and audit service

External services

- Maps / geolocation API
- Push notification provider
- SMS provider

Data storage

- Relational database for users, students, trips, routes, notifications
- Optional real-time database / cache for live bus locations

3.2 Technology stack

Mobile app: Flutter

Backend: NestJS

Database: MySQL

Real-time updates: WebSockets

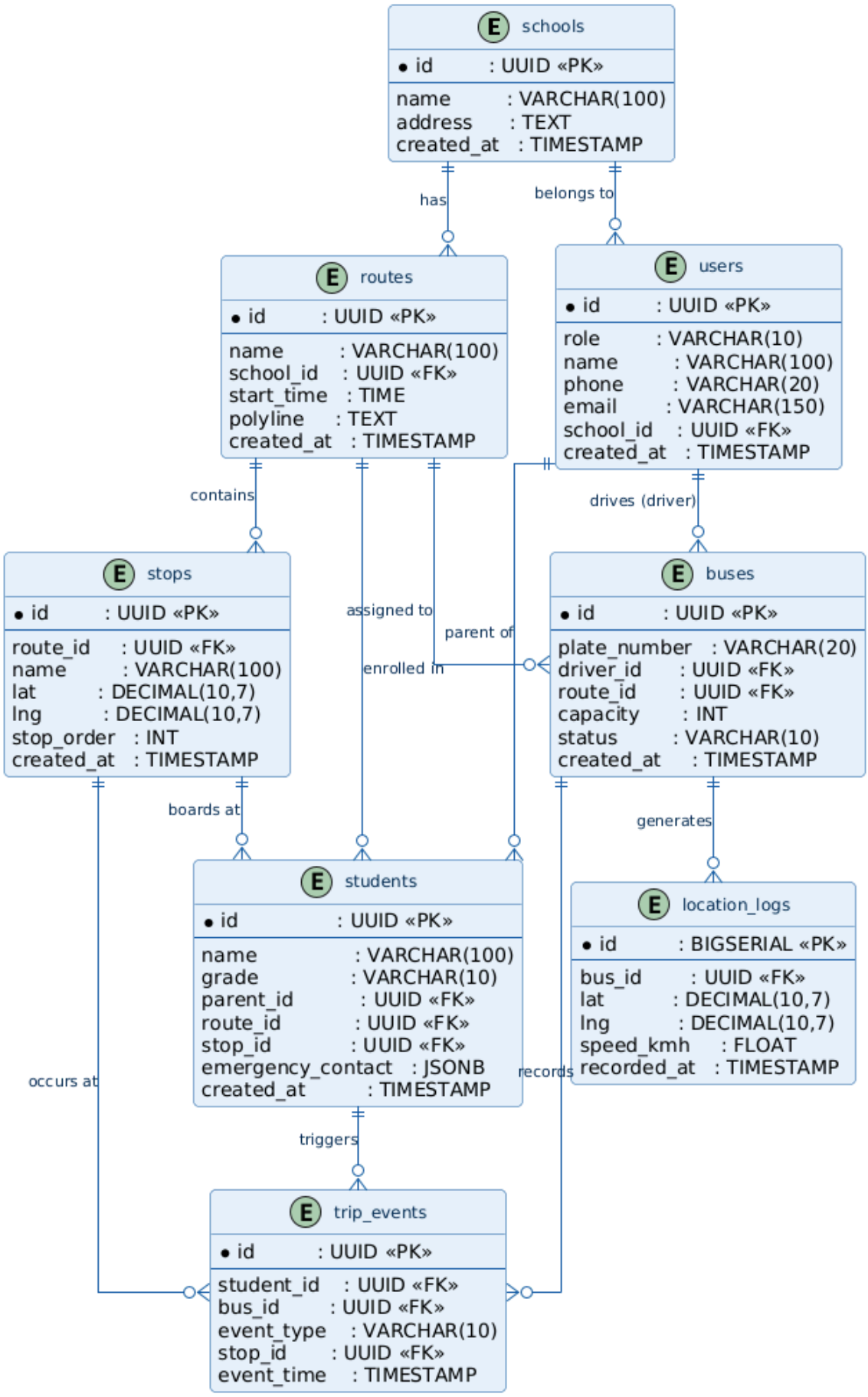
Maps: Google Maps SDK

Push notifications: Firebase Cloud Messaging

SMS fallback gateway: Twilio

3.3 Database schema

Entity Relationship (ER) Diagram for SafeRide


schools
• id : UUID «PK»
name : VARCHAR(100)
address : TEXT
created_at : TIMESTAMP
has
belongs to
routes
• id : UUID «PK»
name : VARCHAR(100)
school_id : UUID «FK»
start_time : TIME
polyline : TEXT
created_at : TIMESTAMP
users
• id : UUID «PK»
role : VARCHAR(10)
name : VARCHAR(100)
phone : VARCHAR(20)
email : VARCHAR(150)
school_id : UUID «FK»
created_at : TIMESTAMP
contains
drives (driver)
stops
• id : UUID «PK»
route_id : UUID «FK»
name : VARCHAR(100)
lat : DECIMAL(10,7)
lng : DECIMAL(10,7)
stop_order : INT
created_at : TIMESTAMP
assigned to
parent of
enrolled in
buses
• id : UUID «PK»
plate_number : VARCHAR(20)
driver_id : UUID «FK»
route_id : UUID «FK»
capacity : INT
status : VARCHAR(10)
created_at : TIMESTAMP
boards at
generates
students
• id : UUID «PK»
name : VARCHAR(100)
grade : VARCHAR(10)
parent_id : UUID «FK»
route_id : UUID «FK»
stop_id : UUID «FK»
emergency_contact : JSONB
created_at : TIMESTAMP
location_logs
• id : BIGSERIAL «PK»
bus_id : UUID «FK»
lat : DECIMAL(10,7)
lng : DECIMAL(10,7)
speed_kmh : FLOAT
recorded_at : TIMESTAMP
occurs at
records
triggers
trip_events
• id : UUID «PK»
student_id : UUID «FK»
bus_id : UUID «FK»
event_type : VARCHAR(10)
stop_id : UUID «FK»
event_time : TIMESTAMP

4. Key mobile features clearly mapped

Phone Feature	How SafeRide Uses It
GPS	Live bus tracking and route visibility
Push notifications	Alerts for approaching bus, boarding, and drop-off
SMS fallback	Backup alerts when internet/app is unavailable
Background task	Continuous location updates during active trips
Contacts integration	Import and manage emergency contacts

5. Conclusion

SafeRide School solves a real and high-impact problem: the lack of visibility and reassurance in daily student transport. By combining real-time GPS tracking, event-based alerts, SMS fallback, background tracking, and emergency contact support, the app improves safety, accountability, and peace of mind for parents, drivers, and school administrators.